

2026-05-07-p2-f24-codex-project-memory

P2-F24 — Codex Project Memory Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development. Steps use checkbox (- []) syntax for tracking.

Programme position: F24 is the **fourth** Phase 2 feature of CLI-Agent Fusion (after F21 Codex Approval Modes, F22 Aider Git Auto-Commit, F23 Cline Browser Tool). Task T01 advances PROGRESS.md from “Phase 2: F23 closed; F24 next candidate (brainstorm)” to “Phase 2 of CLI-Agent Fusion programme: F24 (Codex Project Memory) in flight” and appends an F24 evidence header to docs/improvements/07_phase_2_evidence.md (already created in F21-T01, extended by F22-T01 + F23-T01).

Goal: Ship a real, end-to-end **project memory** subsystem for the HelixCode CLI agent. F24 ADDS a NEW internal/projectmemory/ package: Memory (immutable value), MemoryLoader (parent-walk discovery + user overlay reader), MemoryRegistry (atomic-pointer Snapshot/Set/Reload), MemoryWatcher (fsnotify wrapper with 200 ms debounce). ADDS a NEW internal/commands/memory_command.go with /memory slash (status / show / edit / reload). MODIFIES BaseAgent to optionally prepend Memory.Render() to the system prompt per-call (nil-safe; existing tests unchanged). MODIFIES cmd/cli/main.go to wire registry + watcher at startup adjacent to F23’s browser block. Discovery filenames: helixcode.md / codex.md / AGENTS.md (first match wins; case-insensitive). User overlay: \$XDG_CONFIG_HOME/helixcode/memory.md (loaded AFTER project memory). 64 KB cap with positive TruncatedProject / TruncatedUser flags. Hot-reload via fsnotify (existing direct dep github.com/fsnotify/fsnotify v1.9.0). Graceful degradation: missing files return empty Memory + nil error; fsnotify watch failure degrades to slash-only reload.

Architecture: New files under helix_code/internal/projectmemory/ — types.go (Memory + MemorySource + sentinels ErrNoMemoryFile / ErrMemoryFileTooLarge + MaxMemoryBytes = 64 * 1024 + DebounceWindow = 200 * time.Millisecond + DiscoveryFilenames = []string{"helixcode.md", "codex.md", "AGENTS.md"}), loader.go (MemoryLoader with Discover(cwd) (Memory, error) + parent-walk + git-root-stop + user-overlay), registry.go (MemoryRegistry with atomic.Pointer[Memory] + Snapshot() lock-free + Set + Reload(ctx) + MemorySnapshotter interface), watcher.go (MemoryWatcher with fsnotify + 200 ms debounce + graceful degradation), doc.go. New internal/commands/memory_command.go for the /memory slash. Two existing files get small additions: cmd/cli/main.go (4 lines: construct loader → registry → call Reload → start watcher → register slash); internal/agent/base_agent.go (3 lines: field memoryRegistry projectmemory.MemorySnapshotter, setter SetMemoryRegistry, prepend in getSystemPrompt when non-nil). Backward-compat: BaseAgent without SetMemoryRegistry is byte-for-byte unchanged.

Tech Stack: Go 1.26, testify v1.11, zap (already direct), fsnotify v1.9.0 (ALREADY direct in helix_code/go.mod). **Zero new external deps** (os, sync, sync/atomic, time, errors, fmt, strings, path/filepath, context, os/exec all stdlib). go mod

tidy after T07 must produce no diff in either go.mod or go.sum. T08's verification step asserts this loudly.

Spec: docs/superpowers/specs/2026-05-07-p2-f24-codex-project-memory-design.md (commit c31b9ac).

Working directory for go commands: helix_code/. Git from meta-repo root.

Anti-bluff smoke (full 4-term applied to F24 surface):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" \
    internal/projectmemory internal/commands/memory_command.go &&
    echo BLUFF || echo clean
```

Must always print clean.

Anti-bluff hot zone: §5.2 of the spec — F24 can degenerate in five ways: (a) discovery returns empty Memory because of case-sensitive filename mismatch on a case-sensitive filesystem (AGENTS.md vs agents.md); (b) hot-reload fsnotify event arrives but the registry is never re-stored because the debounce timer is reset infinitely under rapid successive events; (c) agent prepended the OLD memory blob because getSystemPrompt cached registry.Snapshot at agent-construct time; (d) loader silently truncates a >64 KB file but TruncatedProject == false (positive evidence missing); (e) missing-file produces an error instead of empty Memory + nil error. The five “what counts as project memory works” criteria — (1) PHASE-A: project file with sentinel MEMORY_FIXTURE_24 is loaded into Memory.Project and User == ""; (2) PHASE-B: missing file returns empty Memory + nil error; (3) PHASE-C: file rewrite triggers fsnotify-driven Reload within 500 ms (Snapshot returns NEW content, NOT old); (4) PHASE-D: project + user overlay loaded together with project before user in render order; (5) PHASE-E: 100 KB file truncated to MaxMemoryBytes with TruncatedProject == true — are each tested with both unit assertions AND a Challenge phase. The Challenge harness uses positive evidence: tempdir-scoped sentinel byte equality (PHASE-A), nil-Memory + nil-error post-condition (PHASE-B), pre/post-Reload byte differential (PHASE-C), Project + User render order byte equality (PHASE-D), TruncatedProject == true + first-64-KB byte equality (PHASE-E). Byte-evidence mismatch is a hard Challenge failure. Absence-of-error is NEVER acceptable.

Why this is consequential: project memory is codex's “AGENTS.md” pattern — without it, the CLI agent has no persistent project context across sessions, and users must re-explain their project's conventions every session. F24 makes the project's helixcode.md (or AGENTS.md) the single source of truth that propagates automatically into every LLM call. F23 (browser) gives the agent eyes; F22 (autocommit) makes its writes visible; F21 (approval) gates user surface; F24 gives the agent persistent project memory. F24's discriminating tests are: (i) PHASE-A's parent-walk discovery against case-insensitive filenames; (ii) PHASE-C's fsnotify hot-reload byte differential within 500 ms; (iii) PHASE-D's project-before-user render order; (iv) PHASE-E's truncation flag; (v) PHASE-B's missing-file-no-error. All five must produce positive evidence; none can be satisfied by absence-of-error.

Task list



P2-F24-T01 — bootstrap F24 evidence section + advance PROGRESS to F24



P2-F24-T02 — `internal/projectmemory/types.go`: `Memory` + `MemorySource` + `sentinels` + `MaxMemoryBytes` + `DebounceWindow` + `DiscoveryFileNames` + `Render()` (TDD)



P2-F24-T03 — `internal/projectmemory/loader.go`: `MemoryLoader.Discover` with `parent-walk` + `git-root-stop` + `case-insensitive match` + `user overlay` + `truncation` (TDD with real tempdirs)



P2-F24-T04 — `internal/projectmemory/registry.go`: `MemoryRegistry` `atomic-pointer Snapshot/Set/Reload` + `MemorySnapshotter` interface (TDD with `-race`)



P2-F24-T05 — `internal/projectmemory/watcher.go`: `MemoryWatcher` `fsnotify` + 200 ms `debounce` + `graceful degrade` (TDD with real `fsnotify` on real tempdir)



P2-F24-T06 — `internal/commands/memory_command.go`: `/memory slash` (`status/show/edit/reload`) with editor `seam` (TDD)



P2-F24-T07 — `internal/agent/base_agent.go` `SetMemoryRegistry` + `getSystemPrompt prepend` + `main.go wiring` + `integration test` (gated)



P2-F24-T08 — Challenge harness 5 phases (`project-only` + `missing-file-graceful` + `hot-reload` + `project-plus-user` + `truncation`) + `close-out` + `push 4 remotes non-force`

Task 1: Bootstrap F24 evidence

Append F24 section header to `docs/improvements/07_phase_2_evidence.md` (created in F21-T01, extended by F22-T01 + F23-T01) with spec SHA `c31b9ac`. Update `PROGRESS.md` current focus from “Phase 2 of CLI-Agent Fusion programme: F23 closed; F24 next candidate (brainstorm)” to “Phase 2 of CLI-Agent Fusion programme: F24 (Codex Project Memory) in flight”. Insert F24 task list (8 items). Verify zero new external deps:

```
cd HelixCode && grep -E "fsnotify" go.mod
# Expected: fsnotify v1.9.0 already present.
git diff go.mod | grep -E "^\\+|^-" | grep -v "^+++\\|^---" &&
    echo "UNEXPECTED" || echo "clean"
```

Update `docs/CONTINUATION.md` root-level mid-flight section with F24 in-flight status.

Commit: `docs(P2-F24-T01): bootstrap F24 evidence + advance PROGRESS to F24 (Codex Project Memory)`.

Task 2: types.go (TDD)

Files: new `helix_code/internal/projectmemory/types.go`, new `helix_code/internal/projectmemory/types_test.go`, new `helix_code/internal/projectmemory/doc.go`.

Define in types.go:- Memory struct { Project, User, ProjectPath, UserPath string; LoadedAt time.Time; TruncatedProject, TruncatedUser bool }.- MemorySource string enum with SourceProject, SourceUser.- Constants: MaxMemoryBytes = 64 * 1024, DebounceWindow = 200 * time.Millisecond.- Variable: DiscoveryFileNames = []string{"helixcode.md", "codex.md", "AGENTS.md"}.- Sentinel errors: ErrNoMemoryFile, ErrMemoryFileTooLarge.- Method: (m Memory) Render() string — Project + "\n\n--- USER MEMORY OVERLAY ---\n\n" + User when both non-empty; just Project when only project; just User when only user; "" when both empty.

Failing tests FIRST:

```
func TestMaxMemoryBytes_Pin(t *testing.T) {
    require.Equal(t, 64*1024, MaxMemoryBytes)
}
func TestDebounceWindow_Pin(t *testing.T) {
    require.Equal(t, 200*time.Millisecond, DebounceWindow)
}
func TestDiscoveryFileNames_OrderPin(t *testing.T) {
    require.Equal(t, []string{"helixcode.md", "codex.md",
        "AGENTS.md"}, DiscoveryFileNames)
}
func TestErrorSentinels_DistinctErrorsIs(t *testing.T) {
    for _, e := range []error{ErrNoMemoryFile,
        ErrMemoryFileTooLarge} {
        wrapped := fmt.Errorf("wrapped: %w", e)
        require.ErrorIs(t, wrapped, e)
    }
}
func TestMemoryRender_Empty(t *testing.T) {
    require.Equal(t, "", Memory{}.Render())
}
func TestMemoryRender_OnlyProject(t *testing.T) {
    require.Equal(t, "p", Memory{Project: "p"}.Render())
}
func TestMemoryRender_OnlyUser(t *testing.T) {
    require.Equal(t, "u", Memory{User: "u"}.Render())
}
func TestMemoryRender_ProjectBeforeUser(t *testing.T) {
    out := Memory{Project: "P", User: "U"}.Render()
    require.True(t, strings.Index(out, "P") < strings.Index(out,
        "U"))
    require.Contains(t, out, "--- USER MEMORY OVERLAY ---")
}
```

Subject: feat(P2-F24-T02): projectmemory types - Memory + Render + sentinels + MaxMemoryBytes + DiscoveryFileNames (TDD).

Task 3: loader.go (TDD)

Files: new helix_code/internal/projectmemory/loader.go, new helix_code/internal/projectmemory/loader_test.go.

```
type MemoryLoader struct {
    xdgConfigHome string
    log            *zap.Logger
}

func NewMemoryLoader(log *zap.Logger) *MemoryLoader {
    xdg := os.Getenv("XDG_CONFIG_HOME")
    if xdg == "" {
        if home, err := os.UserHomeDir(); err == nil {
            xdg = filepath.Join(home, ".config")
        }
    }
    if log == nil { log = zap.NewNop() }
    return &MemoryLoader{xdgConfigHome: xdg, log: log}
}

func (l *MemoryLoader) Discover(cwd string) (Memory, error) {
    projPath, err := l.findProjectMemory(cwd)
    if err != nil { return Memory{}, err }
    var project string
    var truncProj bool
    if projPath != "" {
        b, rerr := os.ReadFile(projPath)
        if rerr != nil { return Memory{},
            fmt.Errorf("projectmemory: read %s: %w", projPath,
                rerr) }
        if len(b) > MaxMemoryBytes {
            b = b[:MaxMemoryBytes]
            truncProj = true
        }
        project = string(b)
    }
    var userPath, user string
    var truncUser bool
    if l.xdgConfigHome != "" {
        userPath = filepath.Join(l.xdgConfigHome, "helixcode",
            "memory.md")
        if b, rerr := os.ReadFile(userPath); rerr == nil {
            if len(b) > MaxMemoryBytes {
                b = b[:MaxMemoryBytes]
                truncUser = true
            }
            user = string(b)
        }
    }
}
```

```

        } else {
            userPath = "" // missing user overlay is not an error
        }
    }
    return Memory{
        Project: project, User: user,
        ProjectPath: projPath, UserPath: userPath,
        LoadedAt: time.Now(),
        TruncatedProject: truncProj, TruncatedUser: truncUser,
    }, nil
}

// findProjectMemory walks parent dirs from cwd looking for the
// first
// matching DiscoveryFileNames entry (case-insensitive). Stops
// at .git
// directory or filesystem root. Returns empty string + nil error
// when
// no match found.
func (l *MemoryLoader) findProjectMemory(start string) (string,
    error)

```

Failing tests FIRST (use real `t.TempDir()` directories — no mock fs):

```

func TestLoader_Discover_FindsHelixcodeMd_AtCwd(t *testing.T) {
    dir := t.TempDir()
    require.NoError(t, os.WriteFile(filepath.Join(dir,
        "helixcode.md"), []byte("FIXTURE_24"), 0644))
    l := NewMemoryLoader(zap.NewNop())
    m, err := l.Discover(dir)
    require.NoError(t, err)
    require.Contains(t, m.Project, "FIXTURE_24")
    require.NotEmpty(t, m.ProjectPath)
}

func TestLoader_Discover_FindsAtParent_ParentWalk(t *testing.T) {
    root := t.TempDir()
    sub := filepath.Join(root, "sub")
    require.NoError(t, os.MkdirAll(sub, 0755))
    require.NoError(t, os.WriteFile(filepath.Join(root,
        "AGENTS.md"), []byte("PARENT_24"), 0644))
    l := NewMemoryLoader(zap.NewNop())
    m, _ := l.Discover(sub)
    require.Contains(t, m.Project, "PARENT_24")
}

func TestLoader_Discover_OrderHelixcodeBeforeCodex(t *testing.T)
{
    dir := t.TempDir()
    require.NoError(t, os.WriteFile(filepath.Join(dir,
        "helixcode.md"), []byte("HELIX_WIN"), 0644))
}

```

```

    require.NoError(t, os.WriteFile(filepath.Join(dir,
        "codex.md"), []byte("CODEX_LOSE"), 0644))
    m, _ := NewMemoryLoader(zap.NewNop()).Discover(dir)
    require.Contains(t, m.Project, "HELIX_WIN")
    require.NotContains(t, m.Project, "CODEX_LOSE")
}

func TestLoader_Discover_StopsAtGitRoot(t *testing.T) {
    root := t.TempDir()
    require.NoError(t, os.MkdirAll(filepath.Join(root, ".git"),
        0755))
    sub := filepath.Join(root, "sub")
    require.NoError(t, os.MkdirAll(sub, 0755))
    grand := filepath.Dir(root) // outside the git root
    if grand != "/" {
        require.NoError(t, os.WriteFile(filepath.Join(grand,
            "AGENTS.md"), []byte("OUTSIDE_LOSE"), 0644))
        defer os.Remove(filepath.Join(grand, "AGENTS.md"))
    }
    m, _ := NewMemoryLoader(zap.NewNop()).Discover(sub)
    require.NotContains(t, m.Project, "OUTSIDE_LOSE")
}

func TestLoader_Discover_MissingFile_NoError(t *testing.T) {
    dir := t.TempDir()
    m, err := NewMemoryLoader(zap.NewNop()).Discover(dir)
    require.NoError(t, err)
    require.Empty(t, m.ProjectPath)
    require.Empty(t, m.Project)
}

func TestLoader_Discover_UserOverlay_LoadedAfterProject(t
    *testing.T) {
    proj := t.TempDir()
    require.NoError(t, os.WriteFile(filepath.Join(proj,
        "helixcode.md"), []byte("PROJ"), 0644))
    xdg := t.TempDir()
    require.NoError(t, os.MkdirAll(filepath.Join(xdg,
        "helixcode"), 0755))
    require.NoError(t, os.WriteFile(filepath.Join(xdg,
        "helixcode", "memory.md"), []byte("USER"), 0644))
    t.Setenv("XDG_CONFIG_HOME", xdg)
    m, _ := NewMemoryLoader(zap.NewNop()).Discover(proj)
    require.Equal(t, "PROJ", m.Project)
    require.Equal(t, "USER", m.User)
    rendered := m.Render()
    require.True(t, strings.Index(rendered, "PROJ") <
        strings.Index(rendered, "USER"))
}

func TestLoader_Discover_TruncatesLargeFile_SetsFlag(t
    *testing.T) {

```

```

    dir := t.TempDir()
    big := strings.Repeat("X", MaxMemoryBytes+100)
    require.NoError(t, os.WriteFile(filepath.Join(dir,
        "helixcode.md"), []byte(big), 0644))
    m, _ := NewMemoryLoader(zap.NewNop()).Discover(dir)
    require.Equal(t, MaxMemoryBytes, len(m.Project))
    require.True(t, m.TruncatedProject)
}

```

Subject: feat(P2-F24-T03): projectmemory loader - parent-walk + git-root-stop + user overlay + truncation (TDD).

Task 4: registry.go (TDD with -race)

Files: new helix_code/internal/projectmemory/registry.go, new helix_code/internal/projectmemory/registry_test.go.

```

type MemorySnapshotter interface {
    Snapshot() Memory
}

type MemoryRegistry struct {
    current atomic.Pointer[Memory]
    loader  *MemoryLoader
    cwd     string
    mu      sync.Mutex
}

func NewMemoryRegistry(loader *MemoryLoader, cwd string)
    *MemoryRegistry {
    return &MemoryRegistry{loader: loader, cwd: cwd}
}

func (r *MemoryRegistry) Snapshot() Memory {
    if p := r.current.Load(); p != nil { return *p }
    return Memory{}
}

func (r *MemoryRegistry) Set(m Memory) {
    r.current.Store(&m)
}

func (r *MemoryRegistry) Reload(ctx context.Context) (Memory,
    error) {
    r.mu.Lock(); defer r.mu.Unlock()
    if r.loader == nil { return Memory{},
        fmt.Errorf("projectmemory: nil loader") }
    m, err := r.loader.Discover(r.cwd)

```



```

    if err != nil { return Memory{}, err } // keep previous on
        err
    r.current.Store(&m)
    return m, nil
}

```

Failing tests FIRST (run with -race in T08 close-out):

```

func TestRegistry_Snapshot_NilCurrent_ZeroValue(t *testing.T) {
    r := NewMemoryRegistry(NewMemoryLoader(zap.NewNop()),
        t.TempDir())
    m := r.Snapshot()
    require.Empty(t, m.Project)
    require.Empty(t, m.ProjectPath)
}

func TestRegistry_Set_Snapshot_Roundtrip(t *testing.T) {
    r := NewMemoryRegistry(NewMemoryLoader(zap.NewNop()),
        t.TempDir())
    r.Set(Memory{Project: "X"})
    require.Equal(t, "X", r.Snapshot().Project)
}

func TestRegistry_Reload_RealTempdir(t *testing.T) {
    dir := t.TempDir()
    require.NoError(t, os.WriteFile(filepath.Join(dir,
        "helixcode.md"), []byte("RELOAD_24"), 0644))
    r := NewMemoryRegistry(NewMemoryLoader(zap.NewNop()), dir)
    m, err := r.Reload(context.Background())
    require.NoError(t, err)
    require.Contains(t, m.Project, "RELOAD_24")
    require.Contains(t, r.Snapshot().Project, "RELOAD_24")
}

func TestRegistry_Reload_KeepsPreviousOnError(t *testing.T) {
    // First reload sets known-good; second reload uses a loader
    // that errs.
    dir := t.TempDir()
    require.NoError(t, os.WriteFile(filepath.Join(dir,
        "helixcode.md"), []byte("GOOD"), 0644))
    r := NewMemoryRegistry(NewMemoryLoader(zap.NewNop()), dir)
    _, _ = r.Reload(context.Background())
    require.Contains(t, r.Snapshot().Project, "GOOD")
    // Replace cwd with a non-existent path; loader.Discover will
    // succeed (returns empty) – so this test
    // verifies that a graceful empty-Memory replaces –
    // DOCUMENTED behaviour. To prove keep-previous
    // we need a loader that errs – use an unreadable file.
    bad := filepath.Join(dir, "helixcode.md")
    require.NoError(t, os.Chmod(bad, 0000))
    defer os.Chmod(bad, 0644)
    if os.Geteuid() == 0 {

```

```

        t.Skip("SKIP-OK: #P2-F24 root bypasses chmod 0000")
    }
    _, err := r.Reload(context.Background())
    require.Error(t, err)
    require.Contains(t, r.Snapshot().Project,
        "GOOD") // previous preserved
}

func TestRegistry_Concurrent_ReadWrite_NoDataRace(t *testing.T) {
    r := NewMemoryRegistry(NewMemoryLoader(zap.NewNop()),
        t.TempDir())
    r.Set(Memory{Project: "INITIAL"})
    var wg sync.WaitGroup
    stop := make(chan struct{})
    for i := 0; i < 5; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for {
                select { case <-stop: return; default: }
                _ = r.Snapshot()
            }
        }()
    }
    for i := 0; i < 100; i++ {
        r.Set(Memory{Project: fmt.Sprintf("V%d", i)})
    }
    close(stop); wg.Wait()
    require.Contains(t, r.Snapshot().Project, "V99")
}

```

Subject: feat(P2-F24-T04): projectmemory registry - atomic-pointer Snapshot/Set/Reload + MemorySnapshotter (TDD -race).

Task 5: watcher.go (TDD with real fsnotify)

Files: new helix_code/internal/projectmemory/watcher.go, new helix_code/internal/projectmemory/watcher_test.go.

```

type MemoryWatcher struct {
    registry *MemoryRegistry
    watcher  *fsnotify.Watcher
    log      *zap.Logger
    debounce time.Duration
    done     chan struct{}
}

func NewMemoryWatcher(r *MemoryRegistry, log *zap.Logger)
    *MemoryWatcher {

```

```

    if log == nil { log = zap.NewNop() }
    return &MemoryWatcher{registry: r, log: log, debounce:
        DebounceWindow, done: make(chan struct{})}
}

func (w *MemoryWatcher) Start(ctx context.Context) error {
    fsw, err := fsnotify.NewWatcher()
    if err != nil { w.log.Warn("projectmemory: fsnotify new
        watcher failed; degrading to slash-only reload"); return
        nil }
    w.watcher = fsw
    snap := w.registry.Snapshot()
    seen := make(map[string]struct{})
    for _, p := range []string{snap.ProjectPath, snap.UserPath} {
        if p == "" { continue }
        parent := filepath.Dir(p)
        if _, dup := seen[parent]; dup { continue }
        seen[parent] = struct{}{}
        if addErr := fsw.Add(parent); addErr != nil {
            w.log.Warn("projectmemory: fsnotify add failed",
                zap.String("dir", parent), zap.Error(addErr))
        }
    }
    go w.runEventLoop(ctx, snap)
    return nil
}

```

```

func (w *MemoryWatcher) runEventLoop(ctx context.Context, snap
    Memory) {
    defer close(w.done)
    var timer *time.Timer
    targets := map[string]struct{}{}
    if snap.ProjectPath != "" { targets[snap.ProjectPath] =
        struct{}{} }
    if snap.UserPath != "" { targets[snap.UserPath] = struct{}{} }
    for {
        select {
        case <-ctx.Done(): return
        case ev, ok := <-w.watcher.Events:
            if !ok { return }
            if _, hit := targets[ev.Name]; !hit { continue }
            if ev.Op&(fsnotify.Write|fsnotify.Create|
                fsnotify.Rename) == 0 { continue }
            if timer != nil { timer.Stop() }
            timer = time.AfterFunc(w.debounce, func() {
                if _, err := w.registry.Reload(ctx); err != nil {

```

```

        w.log.Warn("projectmemory: reload after
fsnotify failed", zap.Error(err))
    }
    })
    case _, ok := <-w.watcher.Errors:
        if !ok { return }
    }
}

func (w *MemoryWatcher) Close() error {
    if w.watcher == nil { return nil }
    err := w.watcher.Close()
    <-w.done
    return err
}

```

Failing tests FIRST (uses real fsnotify on real tempdir):

```

func TestWatcher_FileWriteTriggersReload_Real(t *testing.T) {
    dir := t.TempDir()
    file := filepath.Join(dir, "helixcode.md")
    require.NoError(t, os.WriteFile(file, []byte("V1"), 0644))
    r := NewMemoryRegistry(NewMemoryLoader(zap.NewNop()), dir)
    _, err := r.Reload(context.Background())
    require.NoError(t, err)
    require.Equal(t, "V1", r.Snapshot().Project)
    w := NewMemoryWatcher(r, zap.NewNop())
    require.NoError(t, w.Start(context.Background()))
    defer w.Close()
    // Rewrite file.
    require.NoError(t, os.WriteFile(file, []byte("V2"), 0644))
    // Wait up to 500 ms.
    deadline := time.Now().Add(500 * time.Millisecond)
    for time.Now().Before(deadline) {
        if r.Snapshot().Project == "V2" { break }
        time.Sleep(20 * time.Millisecond)
    }
    require.Equal(t, "V2", r.Snapshot().Project)
}

func TestWatcher_Close_Idempotent(t *testing.T) {
    dir := t.TempDir()
    require.NoError(t, os.WriteFile(filepath.Join(dir,
        "helixcode.md"), []byte("X"), 0644))
    r := NewMemoryRegistry(NewMemoryLoader(zap.NewNop()), dir)
    _, _ = r.Reload(context.Background())
    w := NewMemoryWatcher(r, zap.NewNop())
    require.NoError(t, w.Start(context.Background()))
    require.NoError(t, w.Close())
}

```

```

    // Second close is a no-op (Close returns whatever
    // fsnotify.Close returns; nil after first).
    _ = w.Close() // no panic
}
func TestWatcher_NoMemoryFile_GracefulStart(t *testing.T) {
    dir := t.TempDir()
    r := NewMemoryRegistry(NewMemoryLoader(zap.NewNop()), dir)
    _, _ = r.Reload(context.Background())
    w := NewMemoryWatcher(r, zap.NewNop())
    require.NoError(t, w.Start(context.Background()))
    defer w.Close()
}

```

Subject: feat(P2-F24-T05): projectmemory watcher - fsnotify + 200ms debounce + graceful degrade (TDD real-fsnotify).

Task 6: memory_command.go (TDD)

Files: new helix_code/internal/commands/memory_command.go, new helix_code/internal/commands/memory_command_test.go.

```

type MemoryCommand struct {
    registry *projectmemory.MemoryRegistry
    editor    func() string // test seam
}

func NewMemoryCommand(r *projectmemory.MemoryRegistry)
    *MemoryCommand {
    return &MemoryCommand{registry: r, editor: func() string {
        if e := os.Getenv("EDITOR"); e != "" { return e }
        return "vi"
    }}
}

func (c *MemoryCommand) Name() string { return "memory" }
func (c *MemoryCommand) Aliases() []string { return nil }
func (c *MemoryCommand) Description() string { return "Inspect,
    show, edit, or reload project memory." }
func (c *MemoryCommand) Usage() string { return "/memory [status|
    show|edit|reload]" }

func (c *MemoryCommand) Execute(ctx context.Context, cc
    *CommandContext) (*CommandResult, error) {
    sub := "status"
    if len(cc.Args) > 0 { sub = cc.Args[0] }
    switch sub {
    case "status":

```

```

        return &CommandResult{Success: true, Output:
            c.handleStatus()}, nil
    case "show":
        return &CommandResult{Success: true, Output:
            c.handleShow()}, nil
    case "edit":
        return c.handleEdit(ctx, cc)
    case "reload":
        return c.handleReload(ctx)
    default:
        return nil, fmt.Errorf("/memory: unknown subcommand %q
            (want status|show|edit|reload)", sub)
    }
}
// handleStatus → "Project path: ... \nProject size: N
//               bytes \nProject truncated: bool \n
//               User path: ... \nUser size: M bytes \nUser
//               truncated: bool \nLoaded at: time".
// handleShow   → m.Render(); empty-message if both empty.
// handleEdit   → resolve path (ProjectPath or filepath.Join(cwd,
//               "helixcode.md"));
//               exec.CommandContext(ctx, c.editor(), path);
//               inherit stdio; run.
// handleReload → registry.Reload(ctx); render new sizes.

```

Failing tests FIRST:

```

func TestMemoryCommand_Name(t *testing.T) {
    require.Equal(t, "memory", NewMemoryCommand(nil).Name())
}
func TestMemoryCommand_Status_NoMemory(t *testing.T) {
    r :=
        projectmemory.NewMemoryRegistry(projectmemory.NewMemoryLoader(zap.
            t.TempDir()))
    cmd := NewMemoryCommand(r)
    res, err := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"status"}})
    require.NoError(t, err)
    require.Contains(t, res.Output, "Project size: 0")
}
func TestMemoryCommand_Show_RendersContent(t *testing.T) {
    dir := t.TempDir()
    require.NoError(t, os.WriteFile(filepath.Join(dir,
        "helixcode.md"), []byte("SHOW_24"), 0644))
    r :=
        projectmemory.NewMemoryRegistry(projectmemory.NewMemoryLoader(zap.
            dir))
    _, _ = r.Reload(context.Background())
    cmd := NewMemoryCommand(r)

```

```

    res, _ := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"show"}})
    require.Contains(t, res.Output, "SHOW_24")
}

func TestMemoryCommand_Edit_RunsEditor(t *testing.T) {
    dir := t.TempDir()
    require.NoError(t, os.WriteFile(filepath.Join(dir,
        "helixcode.md"), []byte("EDIT_24"), 0644))
    r :=
        projectmemory.NewMemoryRegistry(projectmemory.NewMemoryLoader(zap.
            dir)
    _, _ = r.Reload(context.Background())
    cmd := NewMemoryCommand(r)
    cmd.editor = func() string { return "true" } // unix true
        exits 0
    res, err := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"edit"}})
    require.NoError(t, err)
    require.True(t, res.Success)
}

func TestMemoryCommand_Reload_RealTempdir(t *testing.T) {
    dir := t.TempDir()
    file := filepath.Join(dir, "helixcode.md")
    require.NoError(t, os.WriteFile(file, []byte("R1"), 0644))
    r :=
        projectmemory.NewMemoryRegistry(projectmemory.NewMemoryLoader(zap.
            dir)
    cmd := NewMemoryCommand(r)
    _, _ = cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"reload"}})
    require.Contains(t, r.Snapshot().Project, "R1")
    require.NoError(t, os.WriteFile(file, []byte("R2"), 0644))
    _, _ = cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"reload"}})
    require.Contains(t, r.Snapshot().Project, "R2")
}

func TestMemoryCommand_UnknownSubcommand_Err(t *testing.T) {
    r :=
        projectmemory.NewMemoryRegistry(projectmemory.NewMemoryLoader(zap.
            t.TempDir())
    cmd := NewMemoryCommand(r)
    _, err := cmd.Execute(context.Background(),
        &CommandContext{Args: []string{"nope"}})
    require.Error(t, err)
}

```

Subject: feat(P2-F24-T06): /memory slash command - status/show/edit/reload + editor seam (TDD).

Task 7: BaseAgent integration + main.go wiring + integration test

Files: modify `helix_code/internal/agent/base_agent.go`, modify `helix_code/cmd/cli/main.go`, new `helix_code/internal/agent/base_agent_memory_test.go`, new `helix_code/tests/integration/memory_test.go` (`//go:build integration`).

`base_agent.go` (MINIMAL additions):

```
import "dev.helix.code/internal/projectmemory"

type BaseAgent struct {
    // ... existing fields ...
    memoryRegistry projectmemory.MemorySnapshotter
}

func (a *BaseAgent) SetMemoryRegistry(r
    projectmemory.MemorySnapshotter) {
    a.memoryRegistry = r
}

func (a *BaseAgent) getSystemPrompt() string {
    base := fmt.Sprintf(`You are a %s agent named %s. Your
        capabilities include: %v.

You are part of a multi-agent system for software
    development. ...`, a.agentType, a.name, a.capabilities)

    if a.memoryRegistry == nil {
        return base
    }
    rendered := a.memoryRegistry.Snapshot().Render()
    if rendered == "" {
        return base
    }
    return rendered + "\n\n---\n\n" + base
}
```

`main.go` (4 lines + defer adjacent to F23 wiring):

```
// F24: project memory subsystem (codex AGENTS.md port).
cwd, _ := os.Getwd()
memLoader := projectmemory.NewMemoryLoader(zap.NewNop())
memRegistry := projectmemory.NewMemoryRegistry(memLoader, cwd)
if _, err := memRegistry.Reload(ctx); err != nil {
    log.Printf("projectmemory: initial reload failed: %v", err)
}
```



```

memWatcher := projectmemory.NewMemoryWatcher(memRegistry,
    zap.NewNop())
if err := memWatcher.Start(ctx); err != nil {

    log.Printf("projectmemory: watcher start failed
        (degrading to slash-only): %v", err)
}
defer memWatcher.Close()
if regErr :=
    cmdRegistry.Register(commands.NewMemoryCommand(memRegistry));
    regErr != nil {
    log.Printf("projectmemory: register slash command failed:
        %v", regErr)
}
c.memoryRegistry = memRegistry // expose to agent constructors

```

Failing tests FIRST:

```

// internal/agent/base_agent_memory_test.go
type stubSnapshotter struct{ m projectmemory.Memory }
func (s *stubSnapshotter) Snapshot() projectmemory.Memory {
    return s.m }

func TestBaseAgent_GetSystemPrompt_NoMemoryRegistry_Unchanged(t
    *testing.T) {
    a := NewBaseAgent("id", "name", AgentTypeOrchestrator, nil)
    out := a.getSystemPrompt()
    require.NotContains(t, out, "USER MEMORY OVERLAY")
}

func TestBaseAgent_GetSystemPrompt_PrependsCurrentMemory(t
    *testing.T) {
    a := NewBaseAgent("id", "name", AgentTypeOrchestrator, nil)
    s := &stubSnapshotter{m: projectmemory.Memory{Project:
        "PROJECT_FIXTURE_24"}}
    a.SetMemoryRegistry(s)
    out := a.getSystemPrompt()
    require.Contains(t, out, "PROJECT_FIXTURE_24")
    // Memory must come before the base prompt's signature line.
    require.True(t, strings.Index(out, "PROJECT_FIXTURE_24") <
        strings.Index(out, "agent named name"))
}

func TestBaseAgent_GetSystemPrompt_LiveSnapshot_NotCached(t
    *testing.T) {
    a := NewBaseAgent("id", "name", AgentTypeOrchestrator, nil)
    s := &stubSnapshotter{m: projectmemory.Memory{Project:
        "OLD_24"}}
    a.SetMemoryRegistry(s)
    require.Contains(t, a.getSystemPrompt(), "OLD_24")
    s.m = projectmemory.Memory{Project: "NEW_24"}
}

```

```

    out := a.getSystemPrompt()
    require.Contains(t, out, "NEW_24")
    require.NotContains(t, out, "OLD_24")
}

```

Failing integration test FIRST:

```
//go:build integration
```

```

func TestMemory_Integration_StartupLoadsProjectFile_Real(t
    *testing.T) {
    dir := t.TempDir()
    require.NoError(t, os.WriteFile(filepath.Join(dir,
        "helixcode.md"), []byte("INTEGRATION_24"), 0644))
    r :=
        projectmemory.NewMemoryRegistry(projectmemory.NewMemoryLoader(zap.
            dir)
    _, err := r.Reload(context.Background())
    require.NoError(t, err)
    require.Contains(t, r.Snapshot().Project, "INTEGRATION_24")
}

func TestMemory_Integration_HotReload_Real(t *testing.T) {
    dir := t.TempDir()
    file := filepath.Join(dir, "helixcode.md")
    require.NoError(t, os.WriteFile(file, []byte("INT_V1"),
        0644))
    r :=
        projectmemory.NewMemoryRegistry(projectmemory.NewMemoryLoader(zap.
            dir)
    _, _ = r.Reload(context.Background())
    w := projectmemory.NewMemoryWatcher(r, zap.NewNop())
    require.NoError(t, w.Start(context.Background()))
    defer w.Close()
    require.NoError(t, os.WriteFile(file, []byte("INT_V2"),
        0644))
    deadline := time.Now().Add(500 * time.Millisecond)
    for time.Now().Before(deadline) {
        if r.Snapshot().Project == "INT_V2" { break }
        time.Sleep(20 * time.Millisecond)
    }
    require.Equal(t, "INT_V2", r.Snapshot().Project)
}

```

Subject: feat(P2-F24-T07): BaseAgent SetMemoryRegistry + main.go wiring + integration test (TDD).

Task 8: Challenge harness 5 phases + close-out + push 4 remotes non-force

Files: new helix_code/tests/integration/cmd/p2f24_challenge/main.go, new challenges/p2-f24-codex-project-memory/CHALLENGE.md, new challenges/p2-f24-codex-project-memory/run.sh.

Harness phases (per spec §6.3):

1. **PHASE-A: PROJECT-ONLY** — tempdir with helixcode.md (sentinel MEMORY_FIXTURE_24); loader.Discover returns Memory with Project containing sentinel, User empty, ProjectPath non-empty.
2. **PHASE-B: MISSING-FILE-GRACEFUL** — empty tempdir; loader.Discover returns empty Memory + nil error; Render() == "".
3. **PHASE-C: HOT-RELOAD** — write MEM_INITIAL_24; start watcher; rewrite to MEM_UPDATED_24; wait up to 500 ms; assert Snapshot returns NEW (not OLD).
4. **PHASE-D: PROJECT-PLUS-USER** — tempdir + \$XDG_CONFIG_HOME overlay; both loaded; Render() contains both with project-before-user order.
5. **PHASE-E: TRUNCATION** — write 100 KB file; loader truncates to MaxMemoryBytes; TruncatedProject == true; first 64 KB byte-equal to input.

Output skeleton ends with:

```
SUMMARY: PHASE-A=4/4 PASS; PHASE-B=4/4 PASS; PHASE-C=3/3 PASS; PHASE-D=3/3  
==> ALL CHECKS PASSED
```

The Challenge MUST exit non-zero on any byte-evidence mismatch. Anti-bluff smoke clean check appended. Verbatim output captured into 07_phase_2_evidence.md. Dual commit (Challenges submodule + meta-repo bump).

challenges/p2-f24-codex-project-memory/run.sh mirrors F22/F23 structure with anti-bluff smoke + cross-compile linux check.

Close-out — tick all 8 items in PROGRESS, advance PROGRESS focus from F24 to “Phase 2 of CLI-Agent Fusion programme: F24 closed; F25 next candidate”. Run final verification:

```
cd HelixCode && go test -count=1 ./internal/projectmemory/...  
go test -count=1 -race ./internal/projectmemory/...  
go test -count=1 ./internal/commands/  
go test -count=1 ./internal/agent/ -run "GetSystemPrompt"  
go test -count=1 -tags=integration -run  
    "TestMemory_Integration" ./tests/integration/...  
grep -rn "simulated\\|for now\\|TODO implement\\|placeholder" \  
    internal/projectmemory internal/commands/memory_command.go &&  
    echo BLUFF || echo clean  
go mod tidy  
git diff --exit-code go.mod go.sum
```

Cross-compile check:

```
cd HelixCode && GOOS=linux GOARCH=amd64 go build -o /tmp/  
helixcode-linux-amd64 ./cmd/server  
ls -la /tmp/helixcode-linux-amd64
```

Commit chore(P2-F24-T08): close out feature 24 — Codex Project Memory. Push 4 remotes non-force (origin, helixdev/github, vasic-digital/upstream, gitlab per programme conventions).

PROGRESS.md milestone entry (verbatim):

- 2026-05-07 — Feature 24 (Codex Project Memory) closed. 8 task commits (T Real, end-to-end project memory subsystem modelled on codex's AGENTS.md pattern. NEW internal/projectmemory/ package: Memory + MemoryLoader (par walk discovery → helixcode.md / codex.md / AGENTS.md, case-insensitive, stops at git root) + MemoryRegistry (atomic.Pointer[Memory] Snapshot/Set Reload + MemorySnapshotter interface) + MemoryWatcher (fsnotify + 200 ms debounce + graceful degrade). NEW /memory slash (status/show/edit/reload BaseAgent.getSystemPrompt prepends Memory.Render() per-call (nil-safe, backward-compatible). User overlay at \$XDG_CONFIG_HOME/helixcode/memory.md loaded AFTER project memory. 64 KB cap with positive TruncatedProject / TruncatedUser flags. Composes with F15 subagents (shared registry per CL instance), F04 worktrees (each worktree's cwd discovers its own memory naturally). Zero new external deps (fsnotify v1.9.0 already direct). Five-phase Challenge harness PASS (PROJECT-ONLY + MISSING-FILE-GRACEFUL HOT-RELOAD + PROJECT-PLUS-USER + TRUNCATION) with positive runtime evidence per Article XI §11.9.

Subject: chore(P2-F24-T08): close out feature 24 — Codex Project Memory.

Self-review notes

1. **Spec coverage:** every spec section maps to a task — T02 types (§3.3), T03 loader (§4.1), T04 registry (§4.2 + concurrency), T05 watcher (§4.3), T06 /memory slash (§3.4), T07 BaseAgent + main.go wiring (§3.5 + §4.1), T08 Challenge 5 phases + close-out (§6.3).
2. **TDD:** every code task starts with failing tests. Loader tests use real `t.TempDir()`. Registry tests run with `-race`. Watcher tests use real `fsnotify` against real `tempdir`.
3. **Type consistency:** Memory, MemorySource, MemoryLoader, MemoryRegistry, MemoryWatcher, MemorySnapshotter, MemoryCommand, sentinel errors (`ErrNoMemoryFile`, `ErrMemoryFileTooLarge`), constants (`MaxMemoryBytes`, `DebounceWindow`, `DiscoveryFilenames`) — all match across spec §3 and plan T02-T08.
4. **Zero new external deps:** `fsnotify v1.9.0` already direct in `helix_code/go.mod`. T08 verifies `git diff --exit-code go.mod go.sum` is no-op.
5. **Anti-bluff (§5.2):** Challenge has 5 phases, each with positive byte-evidence: `tempdir-scoped` sentinel byte equality (PHASE-A), `nil-Memory + nil-error` (PHASE-B), `pre/post-Reload` byte differential within 500 ms (PHASE-C), `Project + User` render order byte equality (PHASE-D), `TruncatedProject == true + first-64-KB` byte equality (PHASE-E). Byte-evidence mismatch is hard failure. Absence-of-error never acceptable.
6. **CONST-042:** memory contents NEVER logged at INFO level. Loader logs only path + byte counts. Per-tool descriptions and telemetry NEVER include memory body.

7. **CONST-043:** F24 emits zero `git push` commands. T08 close-out push to 4 remotes requires explicit user authorisation per push.
8. **CONST-033:** F24 emits no shell commands beyond `/memory edit`'s `$EDITOR` invocation (runtime user-space process, not a power-state transition).
9. **F15 subagents integration:** registry is shared per-CLI-instance; subagents see the same memory. F15 task-specific `Input` carries per-subagent context — separate from project memory.
10. **F04 worktree integration:** each worktree's `cwd` is its own discovery root; worktrees naturally inherit their parent project's memory via parent-walk; no special handling needed.
11. **Backward compat:** `BaseAgent` without `SetMemoryRegistry` is byte-for-byte unchanged. Existing tests in `base_agent_extended_test.go` continue to pass.
12. **Non-obvious call: parent-dir fsnotify watch (not file-level)** — atomic-write editors (vim, emacs) replace the file's inode; file-level watches lose subscription. Parent-dir watch survives.
13. **Non-obvious call: 200 ms debounce window** — vim atomic-write produces 3-5 events in ~50 ms; 200 ms coalesces into one `Reload`.
14. **Non-obvious call: project before user in render order** — security mandate; user-level overrides cannot bypass project-level rules.
15. **Non-obvious call: missing files are NOT errors** — `Discover` returns empty `Memory + nil`. Missing project memory is a normal user state (most projects don't have one yet).
16. **Non-obvious call: vi fallback (not editor or nano)** — POSIX-mandatory; portable across all unix variants.
17. **Non-obvious call: 64 KB cap matches F23's MaxSnapshotBytes** — symmetry across the codebase.
18. **Non-obvious call: MemorySnapshotter interface not concrete *MemoryRegistry** — `BaseAgent` depends on read-only contract; test fakes implement only `Snapshot()` `Memory`.
19. **Non-obvious call: registry.Set is exposed (not just Reload)** — useful for tests that want to inject `Memory` directly without disk I/O.
20. **Non-obvious call: case-insensitive matching via two-pass dir scan** — fast on case-insensitive FS, fallback for case-sensitive.
21. **Fourth Phase 2 feature:** F24 is the fourth Phase 2 feature after F21 + F22 + F23. T01 advances `PROGRESS.md` from "F23 closed; F24 next candidate" to "F24 in flight"; appends F24 evidence header to existing `07_phase_2_evidence.md`.